MACHINE LEARNING LAB RECORD

Name:G Deevena

Reg.No:192311391

Course Code:ITA0614

Course Name: Machine Learning

1.FIND-S ALGORITHM
MACHINE LEARNING LAB RECORD

The FIND-S algorithm finds the most specific hypothesis consistent with the positive training examples. For each positive example, differing attribute values are generalized to "?". Negative examples are ignored.

Problem 1: Enjoying Sport

Aim

To implement the FIND-S algorithm to find the most specific hypothesis for predicting whether a person enjoys playing sport based on weather conditions.

Algorithm

1. Start the program.
2. Initialize the hypothesis as empty.
3. Read the training dataset.
4. For each positive example, use the first positive example as the initial hypothesis.
5. For each subsequent positive example, replace any differing attribute value with '?'.
6. Ignore negative examples.
7. Display the final hypothesis.
8. Stop the program.

Python Code

```
# FIND-S Algorithm: Enjoying Sport
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'No'],
    ['Sunny', 'Warm', 'High', 'Weak', 'Yes']
]

h = None
for row in data:
    if row[-1] == 'Yes':
        if h is None:
            h = row[:-1].copy()
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
```

```
                h[i] = '?'

print("Final Hypothesis:", h)
```

Sample Input

Sky	Temperature	Humidity	Wind	Enjoy Sport
Sunny	Warm	Normal	Strong	Yes
Sunny	Warm	High	Strong	Yes
Rainy	Cold	High	Strong	No
Sunny	Warm	High	Weak	Yes

Sample Output

Final Hypothesis: ['Sunny', 'Warm', '?', '?']

Result

Thus, the FIND-S algorithm was implemented successfully, and the final hypothesis for enjoying sport was obtained as ['Sunny', 'Warm', '?', '?'].

Problem 2: Buying a Computer

Aim

To implement the FIND-S algorithm to identify the most specific hypothesis for predicting whether a person will buy a computer based on age, income, and student status.

Algorithm

9. Start the program.
10. Initialize the hypothesis as empty.
11. Read the training dataset.
12. For each positive example, use the first positive example as the initial hypothesis.
13. For each subsequent positive example, replace any differing attribute value with '?'.
14. Ignore negative examples.
15. Display the final hypothesis.
16. Stop the program.

Python Code

```
# FIND-S Algorithm: Buying a Computer
data = [
    ['Young', 'High', 'No', 'Yes'],
    ['Young', 'Medium', 'Yes', 'Yes'],
    ['Middle', 'High', 'No', 'No'],
    ['Young', 'Low', 'Yes', 'Yes']
]

h = None
for row in data:
    if row[-1] == 'Yes':
        if h is None:
            h = row[:-1].copy()
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
```

```
print("Final Hypothesis:", h)
```

Sample Input

Age	Income	Student	Buy Computer
Young	High	No	Yes
Young	Medium	Yes	Yes
Middle	High	No	No
Young	Low	Yes	Yes

Sample Output

Final Hypothesis: ['Young', '?', '?']

Result

Thus, the FIND-S algorithm was implemented successfully, and the final hypothesis for buying a computer was obtained as ['Young', '?', '?'].

Problem 3: Enjoying a Movie

Aim

To implement the FIND-S algorithm to find the most specific hypothesis for predicting whether a person enjoys a movie based on its genre, language, and duration.

Algorithm

17. Start the program.
18. Initialize the hypothesis as empty.
19. Read the training dataset.
20. For each positive example, use the first positive example as the initial hypothesis.
21. For each subsequent positive example, replace any differing attribute value with '?'.
22. Ignore negative examples.
23. Display the final hypothesis.
24. Stop the program.

Python Code

```
# FIND-S Algorithm: Enjoying a Movie
data = [
    ['Action', 'English', 'Short', 'Yes'],
    ['Action', 'Tamil', 'Short', 'Yes'],
    ['Comedy', 'English', 'Long', 'No'],
    ['Action', 'English', 'Long', 'Yes']
]

h = None
for row in data:
    if row[-1] == 'Yes':
        if h is None:
            h = row[:-1].copy()
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'

print("Final Hypothesis:", h)
```

Sample Input

Genre	Language	Duration	Enjoy Movie
Action	English	Short	Yes
Action	Tamil	Short	Yes
Comedy	English	Long	No
Action	English	Long	Yes

Sample Output

Final Hypothesis: ['Action', '?', '?']

Result

Thus, the FIND-S algorithm was implemented successfully, and the final hypothesis for enjoying a movie was obtained as ['Action', '?', '?'].

Problem 4: Selecting a Mobile Phone

Aim

To implement the FIND-S algorithm to find the most specific hypothesis for predicting whether a customer will buy a mobile phone based on brand, battery capacity, and price.

Algorithm

25. Start the program.
26. Initialize the hypothesis as empty.
27. Read the training dataset.
28. For each positive example, use the first positive example as the initial hypothesis.
29. For each subsequent positive example, replace any differing attribute value with '?'.
30. Ignore negative examples.
31. Display the final hypothesis.
32. Stop the program.

Python Code

```
# FIND-S Algorithm: Selecting a Mobile Phone
data = [
    ['Samsung', 'High', 'Medium', 'Yes'],
    ['Samsung', 'High', 'Low', 'Yes'],
    ['Apple', 'Low', 'High', 'No'],
    ['Samsung', 'Medium', 'High', 'Yes']
]

h = None
for row in data:
    if row[-1] == 'Yes':
        if h is None:
            h = row[:-1].copy()
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'

print("Final Hypothesis:", h)
```

Sample Input

Brand	Battery	Price	Buy Phone
Samsung	High	Medium	Yes
Samsung	High	Low	Yes
Apple	Low	High	No
Samsung	Medium	High	Yes

Sample Output

Final Hypothesis: ['Samsung', '?', '?']

Result

Thus, the FIND-S algorithm was implemented successfully, and the final hypothesis for selecting a mobile phone was obtained as ['Samsung', '?', '?'].

Problem 5: Travelling by Bus

Aim

To implement the FIND-S algorithm to determine the most specific hypothesis for predicting whether a person will travel by bus based on distance, traffic conditions, and ticket price.

Algorithm

33. Start the program.
34. Initialize the hypothesis as empty.
35. Read the training dataset.
36. For each positive example, use the first positive example as the initial hypothesis.
37. For each subsequent positive example, replace any differing attribute value with '?'.
38. Ignore negative examples.
39. Display the final hypothesis.
40. Stop the program.

Python Code

```
# FIND-S Algorithm: Travelling by Bus
data = [
    ['Short', 'Low', 'Cheap', 'Yes'],
    ['Short', 'High', 'Cheap', 'Yes'],
    ['Long', 'High', 'Expensive', 'No'],
    ['Short', 'Medium', 'Expensive', 'Yes']
]

h = None
for row in data:
    if row[-1] == 'Yes':
        if h is None:
            h = row[:-1].copy()
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'

print("Final Hypothesis:", h)
```

Sample Input

Distance	Traffic	Ticket Price	Travel by Bus
Short	Low	Cheap	Yes
Short	High	Cheap	Yes
Long	High	Expensive	No
Short	Medium	Expensive	Yes

Sample Output

Final Hypothesis: ['Short', '?', '?']

Result

Thus, the FIND-S algorithm was implemented successfully, and the final hypothesis for travelling by bus was obtained as ['Short', '?', '?'].

Quick Summary

Problem	Final Hypothesis
1: Enjoying Sport	['Sunny', 'Warm', '?', '?']
2: Buying a Computer	['Young', '?', '?']
3: Enjoying a Movie	['Action', '?', '?']
4: Selecting a Mobile Phone	['Samsung', '?', '?']
5: Travelling by Bus	['Short', '?', '?']

Note: The dataset is embedded in each program, so it serves as the sample input. The question mark (?) means that any value is accepted for that attribute.

2.CANDIDATE-ELIMINATION ALGORITHM

Candidate-Elimination maintains S (the most specific boundary) and G (the most general boundary). This document uses a finite categorical hypothesis space: each attribute is either one of its values observed in the CSV or ?. The program enumerates all such hypotheses and outputs the complete version space and its boundaries.

Problem 1: Enjoying Sport

Aim

To implement Candidate-Elimination using CSV training examples and find hypotheses consistent with whether a person enjoys sport.

Algorithm

41. Start the program and load the CSV file.
42. Identify the attribute columns and the final class-label column.
43. Generate the categorical hypothesis space using observed attribute values and the wildcard ?.
44. For each positive example, keep only hypotheses that cover the example.
45. For each negative example, remove hypotheses that cover the example.
46. Find the most-specific and most-general boundary members of the remaining version space.
47. Display S, G, and all consistent hypotheses; stop.

CSV Sample Input

Save the following text as sport.csv:

```
Sky,Temperature,Humidity,EnjoySport
Sunny,Warm,Normal,Yes
Sunny,Warm,High,Yes
Rainy,Cold,High,No
Sunny,Warm,High,Yes
```

Python Code

```
import csv
from itertools import product

def covers(h, x):
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general_or_equal(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def candidate_elimination(filename):
    with open(filename, newline="") as f:
        rows = list(csv.reader(f))

    headers = rows[0]
    examples = rows[1:]
    X = [row[:-1] for row in examples]
    y = [row[-1].strip().lower() for row in examples]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(headers) - 1)]

    # Finite hypothesis space: observed attribute values or '?'
    hypotheses = list(product(*[[*domain, "?"] for domain in domains]))
    version_space = set(hypotheses)

    for x, label in zip(X, y):
        if label in ("yes", "1", "true", "positive"):
            version_space = {h for h in version_space if covers(h, x)}
        else:
            version_space = {h for h in version_space if not covers(h, x)}
```

```
    if not version_space:
        print("Version space is empty.")
        return

    S = [h for h in version_space
         if not any(more_general_or_equal(h2, h) and h2 != h
                    for h2 in version_space)]
    G = [h for h in version_space
         if not any(more_general_or_equal(h, h2) and h2 != h
                    for h2 in version_space)]

    print("Specific boundary S:")
    for h in sorted(S):
        print(" ", h)
    print("General boundary G:")
    for h in sorted(G):
        print(" ", h)
    print("All consistent hypotheses (version space):")
    for h in sorted(version_space):
        print(" ", h)

if __name__ == "__main__":
    candidate_elimination("sport.csv")
```

Sample Output

```
Specific boundary S:
 ('?', 'Warm', '?')
 ('Sunny', '?', '?')
General boundary G:
 ('Sunny', 'Warm', '?')
All consistent hypotheses (version space):
 ('?', 'Warm', '?')
 ('Sunny', '?', '?')
 ('Sunny', 'Warm', '?')
```

Result

Thus, Candidate-Elimination was demonstrated using sport.csv. The program reads examples from the CSV file and outputs the complete version space under the stated categorical hypothesis representation, along with its S and G boundaries.

Problem 2: Buying a Computer

Aim

To implement Candidate-Elimination using a CSV dataset and find hypotheses consistent with whether a person buys a computer.

Algorithm

48. Start the program and load the CSV file.
49. Identify the attribute columns and the final class-label column.
50. Generate the categorical hypothesis space using observed attribute values and the wildcard ?.
51. For each positive example, keep only hypotheses that cover the example.
52. For each negative example, remove hypotheses that cover the example.
53. Find the most-specific and most-general boundary members of the remaining version space.
54. Display S, G, and all consistent hypotheses; stop.

CSV Sample Input

Save the following text as computer.csv:

```
Age,Income,Student,BuyComputer
Young,High,No,Yes
```

Young,Medium,Yes,Yes
Middle,High,No,No
Young,Low,Yes,Yes

Python Code

```
import csv
from itertools import product

def covers(h, x):
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general_or_equal(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def candidate_elimination(filename):
    with open(filename, newline="") as f:
        rows = list(csv.reader(f))

    headers = rows[0]
    examples = rows[1:]
    X = [row[:-1] for row in examples]
    y = [row[-1].strip().lower() for row in examples]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(headers) - 1)]

    # Finite hypothesis space: observed attribute values or '?'
    hypotheses = list(product(*[[*domain, "?"] for domain in domains]))
    version_space = set(hypotheses)

    for x, label in zip(X, y):
        if label in ("yes", "1", "true", "positive"):
            version_space = {h for h in version_space if covers(h, x)}
        else:
            version_space = {h for h in version_space if not covers(h, x)}

    if not version_space:
        print("Version space is empty.")
        return

    S = [h for h in version_space
         if not any(more_general_or_equal(h2, h) and h2 != h
                    for h2 in version_space)]
    G = [h for h in version_space
         if not any(more_general_or_equal(h, h2) and h2 != h
                    for h2 in version_space)]

    print("Specific boundary S:")
    for h in sorted(S):
        print(" ", h)
    print("General boundary G:")
    for h in sorted(G):
        print(" ", h)
    print("All consistent hypotheses (version space):")
    for h in sorted(version_space):
        print(" ", h)

if __name__ == "__main__":
    candidate_elimination("computer.csv")
```

Sample Output

```
Specific boundary S:
  ('Young', '?', '?')
General boundary G:
  ('Young', '?', '?')
All consistent hypotheses (version space):
  ('Young', '?', '?')
```

Result

Thus, Candidate-Elimination was demonstrated using computer.csv. The program reads examples from the CSV file and outputs the complete version space under the stated categorical hypothesis representation, along with its S and G boundaries.

Problem 3: Enjoying a Movie

Aim

To implement Candidate-Elimination using a CSV file and identify the version space for predicting whether a person enjoys a movie.

Algorithm

55. Start the program and load the CSV file.
56. Identify the attribute columns and the final class-label column.
57. Generate the categorical hypothesis space using observed attribute values and the wildcard ?.
58. For each positive example, keep only hypotheses that cover the example.
59. For each negative example, remove hypotheses that cover the example.
60. Find the most-specific and most-general boundary members of the remaining version space.
61. Display S, G, and all consistent hypotheses; stop.

CSV Sample Input

Save the following text as movie.csv:

```
Genre,Language,Duration,EnjoyMovie
Action,English,Short,Yes
Action,Tamil,Short,Yes
Comedy,English,Long,No
Action,English,Long,Yes
```

Python Code

```
import csv
from itertools import product

def covers(h, x):
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general_or_equal(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def candidate_elimination(filename):
    with open(filename, newline="") as f:
        rows = list(csv.reader(f))

    headers = rows[0]
    examples = rows[1:]
    X = [row[:-1] for row in examples]
    y = [row[-1].strip().lower() for row in examples]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(headers) - 1)]

    # Finite hypothesis space: observed attribute values or '?'
    hypotheses = list(product(*[[*domain, "?"] for domain in domains]))
    version_space = set(hypotheses)

    for x, label in zip(X, y):
        if label in ("yes", "1", "true", "positive"):
            version_space = {h for h in version_space if covers(h, x)}
        else:
            version_space = {h for h in version_space if not covers(h, x)}

    if not version_space:
        print("Version space is empty.")
        return

    S = [h for h in version_space
         if not any(more_general_or_equal(h2, h) and h2 != h
                    for h2 in version_space)]
```

```
    G = [h for h in version_space
         if not any(more_general_or_equal(h, h2) and h2 != h
                    for h2 in version_space)]

    print("Specific boundary S:")
    for h in sorted(S):
        print(" ", h)
    print("General boundary G:")
    for h in sorted(G):
        print(" ", h)
    print("All consistent hypotheses (version space):")
    for h in sorted(version_space):
        print(" ", h)

if __name__ == "__main__":
    candidate_elimination("movie.csv")
```

Sample Output

```
Specific boundary S:
  ('Action', '?', '?')
General boundary G:
  ('Action', '?', '?')
All consistent hypotheses (version space):
  ('Action', '?', '?')
```

Result

Thus, Candidate-Elimination was demonstrated using movie.csv. The program reads examples from the CSV file and outputs the complete version space under the stated categorical hypothesis representation, along with its S and G boundaries.

Problem 4: Selecting a Mobile Phone

Aim

To implement Candidate-Elimination using CSV training data and determine the hypotheses consistent with mobile-phone purchase examples.

Algorithm

62. Start the program and load the CSV file.
63. Identify the attribute columns and the final class-label column.
64. Generate the categorical hypothesis space using observed attribute values and the wildcard ?.
65. For each positive example, keep only hypotheses that cover the example.
66. For each negative example, remove hypotheses that cover the example.
67. Find the most-specific and most-general boundary members of the remaining version space.
68. Display S, G, and all consistent hypotheses; stop.

CSV Sample Input

Save the following text as mobile.csv:

```
Brand,Battery,Price,BuyPhone
Samsung,High,Medium,Yes
Samsung,High,Low,Yes
Apple,Low,High,No
Samsung,Medium,High,Yes
```

Python Code

```
import csv
from itertools import product

def covers(h, x):
```

```
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general_or_equal(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def candidate_elimination(filename):
    with open(filename, newline="") as f:
        rows = list(csv.reader(f))

    headers = rows[0]
    examples = rows[1:]
    X = [row[:-1] for row in examples]
    y = [row[-1].strip().lower() for row in examples]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(headers) - 1)]

    # Finite hypothesis space: observed attribute values or '?'
    hypotheses = list(product(*[[*domain, "?"] for domain in domains]))
    version_space = set(hypotheses)

    for x, label in zip(X, y):
        if label in ("yes", "1", "true", "positive"):
            version_space = {h for h in version_space if covers(h, x)}
        else:
            version_space = {h for h in version_space if not covers(h, x)}

    if not version_space:
        print("Version space is empty.")
        return

    S = [h for h in version_space
         if not any(more_general_or_equal(h2, h) and h2 != h
                    for h2 in version_space)]
    G = [h for h in version_space
         if not any(more_general_or_equal(h, h2) and h2 != h
                    for h2 in version_space)]

    print("Specific boundary S:")
    for h in sorted(S):
        print(" ", h)
    print("General boundary G:")
    for h in sorted(G):
        print(" ", h)
    print("All consistent hypotheses (version space):")
    for h in sorted(version_space):
        print(" ", h)

if __name__ == "__main__":
    candidate_elimination("mobile.csv")
```

Sample Output

```
Specific boundary S:
  ('Samsung', '?', '?')
General boundary G:
  ('Samsung', '?', '?')
All consistent hypotheses (version space):
  ('Samsung', '?', '?')
```

Result

Thus, Candidate-Elimination was demonstrated using mobile.csv. The program reads examples from the CSV file and outputs the complete version space under the stated categorical hypothesis representation, along with its S and G boundaries.

Problem 5: Travelling by Bus

Aim

To implement Candidate-Elimination using a CSV file and find the version space for predicting whether a person travels by bus.

Algorithm

69. Start the program and load the CSV file.
70. Identify the attribute columns and the final class-label column.
71. Generate the categorical hypothesis space using observed attribute values and the wildcard ?.
72. For each positive example, keep only hypotheses that cover the example.
73. For each negative example, remove hypotheses that cover the example.
74. Find the most-specific and most-general boundary members of the remaining version space.
75. Display S, G, and all consistent hypotheses; stop.

CSV Sample Input

Save the following text as bus.csv:

```
Distance,Traffic,TicketPrice,TravelByBus
Short,Low,Cheap,Yes
Short,High,Cheap,Yes
Long,High,Expensive,No
Short,Medium,Expensive,Yes
```

Python Code

```
import csv
from itertools import product

def covers(h, x):
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general_or_equal(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def candidate_elimination(filename):
    with open(filename, newline="") as f:
        rows = list(csv.reader(f))

    headers = rows[0]
    examples = rows[1:]
    X = [row[:-1] for row in examples]
    y = [row[-1].strip().lower() for row in examples]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(headers) - 1)]

    # Finite hypothesis space: observed attribute values or '?'
    hypotheses = list(product(*[[*domain, "?"] for domain in domains]))
    version_space = set(hypotheses)

    for x, label in zip(X, y):
        if label in ("yes", "1", "true", "positive"):
            version_space = {h for h in version_space if covers(h, x)}
        else:
            version_space = {h for h in version_space if not covers(h, x)}

    if not version_space:
        print("Version space is empty.")
        return

    S = [h for h in version_space
         if not any(more_general_or_equal(h2, h) and h2 != h
                    for h2 in version_space)]
    G = [h for h in version_space
         if not any(more_general_or_equal(h, h2) and h2 != h
                    for h2 in version_space)]

    print("Specific boundary S:")
    for h in sorted(S):
        print(" ", h)
    print("General boundary G:")
    for h in sorted(G):
        print(" ", h)
    print("All consistent hypotheses (version space):")
    for h in sorted(version_space):
        print(" ", h)
```

```
if __name__ == "__main__":
    candidate_elimination("bus.csv")
```

Sample Output

```
Specific boundary S:
  ('Short', '?', '?')
General boundary G:
  ('Short', '?', '?')
All consistent hypotheses (version space):
  ('Short', '?', '?')
```

Result

Thus, Candidate-Elimination was demonstrated using bus.csv. The program reads examples from the CSV file and outputs the complete version space under the stated categorical hypothesis representation, along with its S and G boundaries.

Important Note

Each dataset is embedded in the code section as a separate CSV sample file. Run the Python code in the same folder as the CSV file. The final column must use Yes for positive examples and No for negative examples. This enumeration-based implementation demonstrates the version space over the finite hypothesis space formed from observed values and ?.